

COMP 3415 – Software Developer
Software Engineering Project
P1 - Requirements and Project Plan

Project Name: Whozin(Whos In)

Repository : <https://github.com/JINAL-1/Whozin>

Group Member:-

Pranavv Patel -

Julia Petterle - 1185004

Jinal Parekh - 1298870

Dean Facho - 1229227

Krishi Patel - 1257190

Project Information	Details
Course	COMP 3415 Software Engineering
Team	Whozin (Who's In)
Repository	https://github.com/JINAL-1/Whozin
Submission	P1 Requirements and Project Plan

1. Stakeholders and User Roles

The following stakeholders and user roles represent the main groups that will use, support, or be affected by Whozin. Each role has different goals and expectations when interacting with the system.

Event Attendee – Primary User:

An event attendee is a Thunder Bay resident, student, or visitor who uses Whozin to discover local events. Attendees expect to be able to browse and search for events, view event details, register for free events or purchase tickets, access their digital tickets, save and share events, and receive relevant event updates and reminders.

Event Organizer – Primary User:

An event organizer may be a business, student organization, community group, or independent organizer that uses Whozin to promote and manage events. Organizers expect to be able to create and publish event listings, provide event and ticket information, update or cancel their events, and communicate important changes to registered attendees.

Content Creator or Promoter – Primary User:

A content creator or promoter is a user who creates event-related posts or short-form content to promote local events. They expect to be able to share event content, promote events to other users, collaborate with event organizers, and interact with the local event community.

Administrator – Primary User:

An administrator is an authorized platform operator responsible for overseeing Whozin. Administrators expect to manage accounts and permissions and oversee events, reports, and platform content to help maintain a safe and trustworthy environment for users.

Payment Service Provider – External Stakeholder:

The payment service provider processes payments for paid event tickets and communicates the payment status back to Whozin. Payment information is handled through the external payment provider so that complete payment-card information does not need to be stored within the Whozin system.

Mapping and Notification Services – External Stakeholder:

External mapping and notification services support features that depend on event locations and communication with users. They are expected to provide services such as event-location information and mobile notification delivery that can be used by Whozin's attendee and organizer features.

Development Team – Project Stakeholder:

The development team is responsible for designing, implementing, testing, documenting, and maintaining Whozin. The team also maintains the project's repository and product backlog as development progresses.

2. Functional Requirements

The following functional requirements describe the major capabilities that Whozin must provide. They are based on the main functionality represented by the project's user stories and will guide the implementation and testing of the system.

FR-01 – User Account and Authentication:

The system shall allow users to create an account, log in using valid credentials, and log out of their account.

FR-02 – Event Creation:

The system shall allow authenticated event organizers to create and publish event listings containing required information such as the event title, description, date, time, location, image, and ticket options.

FR-03 – Event Management:

The system shall allow event organizers to edit the details of events they have created and cancel their events when necessary.

FR-04 – Personalized Event Feed:

The system shall provide logged-in users with a personalized event feed that recommends events based on information such as their interests, location, previous activity, and engagement. For users without previous activity, the system shall provide popular and nearby events.

FR-05 – Event Search and Filtering:

The system shall allow users to search for events by keyword or location and filter the results using criteria including date, category, distance, and price.

FR-06 – Free Event Registration:

The system shall allow authenticated users to register for available free events and shall create a registration and digital ticket when the registration is successfully completed.

FR-07 – Paid Ticket Purchasing:

The system shall allow authenticated users to select tickets for paid events, view the total purchase price, and securely complete payment through Stripe. Tickets shall only be issued after the payment has been successfully confirmed.

FR-08 – Digital Ticket Access:

The system shall allow users to view their upcoming and past event tickets. Each valid ticket shall display the associated event information and a unique QR code that can be presented for entry.

FR-09 – Registration Cancellation and Refunds:

The system shall allow users to cancel eligible upcoming event registrations. When applicable under the event's refund policy, the system shall process a refund for a paid ticket and release the cancelled spot.

FR-10 – Event Sharing:

The system shall allow users to share an event by generating a shareable link and providing supported in-app or external sharing options.

FR-11 – Calendar Integration:

The system shall allow users to add a registered or saved event to a supported calendar with the correct event title, date, time, and location.

FR-12 – Notifications and Reminders:

The system shall send users appropriate notifications for important event activity, including registration or purchase confirmations, upcoming event reminders, and changes or cancellations made by event organizers.

3. Non-Functional Requirements

The following 10 non-functional requirements describe the expected quality and operating standards of Whozin. Each requirement is specific enough to be checked through testing or inspection.

NFR-01 – Performance:

Under normal operating conditions, at least 95% of requests for the event feed, search results, event details, account pages, and ticket pages shall return a usable response within two seconds. Time spent waiting for an external service such as Stripe is not included in this limit.

Verification: Response times will be measured during testing using application logs and performance-testing tools.

NFR-02 – Scalability:

The system shall support at least 100 users browsing, searching, and viewing events at the same time without exceeding the two-second response-time target defined in NFR-01.

Verification: The team will perform a load test in the testing environment using simulated users.

NFR-03 – Security:

All information sent between the Whozin application and its server shall use HTTPS. Passwords shall be securely hashed and never stored as plain text. The server shall also check a user's role before allowing access to organizer or administrator functions.

Verification: The team will inspect the security configuration, database records, and role-protected endpoints.

NFR-04 – Payment Privacy:

Whozin shall use Stripe to process payments and shall not store complete card numbers, security codes, or other raw payment information in its database or application logs.

Verification: The payment implementation and stored database information will be reviewed before paid ticketing is enabled.

NFR-05 – Reliability:

A successful registration or payment shall create only one valid ticket for each purchased or reserved spot. Repeated requests or payment confirmations shall not create duplicate orders, and the number of confirmed tickets shall not exceed the event's available capacity.

Verification: The team will test repeated registration requests, duplicate payment confirmations, and attempts to register for a full event.

NFR-06 – Usability:

During usability testing, a first-time user shall be able to create an account, find a specified event, and complete a free-event registration in no more than three minutes without assistance.

Verification: This process will be tested with at least five participants who are not directly involved in developing the tested feature.

NFR-07 – Accessibility:

The main account, event-discovery, event-details, registration, checkout, and ticket screens shall follow WCAG 2.1 Level AA guidelines for colour contrast, readable text, labelled form fields, text alternatives, and accessible navigation.

Verification: The team will use an accessibility checklist, automated scanning tools, and manual testing of the main user flows.

NFR-08 – Compatibility and Portability:

The mobile application shall support the main user flows on Android 11 or later and iOS 16 or later. Screens shall remain readable and usable on common phone screen sizes without text, buttons, or forms overlapping or being cut off.

Verification: The team will test the application using Android and iOS devices or emulators with different screen sizes.

NFR-09 – Maintainability:

Important backend functions and frontend services shall have automated tests covering at least 70% of their executable code. All code submitted through a pull request shall pass the project's automated tests before it is merged into the main branch.

Verification: Test reports and GitHub pull-request checks will be reviewed throughout development.

NFR-10 – Backup and Recovery:

The Whozin database shall be backed up at least once every 24 hours when the system is in active use. The team shall also test and document the steps required to restore event, account, registration, and ticket information from a backup.

Verification: The team will keep a record of database backups and complete at least one successful restoration test before the final release.

Together, these requirements provide clear standards for how Whozin should perform beyond simply offering its planned features. They will help the team evaluate whether the application is fast, secure, reliable, easy to use, and compatible with the intended mobile devices. These

requirements may be reviewed and adjusted as the project develops, but any changes should remain measurable and consistent with the system's main functions.

4. User Stories and Acceptance Criteria

US-01 – Create Account and Log In

User Story: As a new user, I want to create an account and log in, so that I can access personalized features like saving events and buying tickets.

Priority: High

Priority Justification: Nearly every other feature (ticketing, feed, organizer tools) requires an authenticated user, so it must exist first.

Expected Deliverable: Working registration, login, and logout functions with secure password handling.

Acceptance Criteria:

- Given a visitor is on the sign-up page, when they submit a valid email and password, then the system creates an account and logs them in.
- Given a registered user enters correct credentials, when they select "Log In", then the system grants access to their account.
- Given a user enters incorrect credentials, when they select "Log In", then the system denies access and shows an error message.

US-02 – Create Event

User Story: As an event organizer, I want to create an event listing with details, so that people can discover and register for it.

Priority: High

Priority Justification: The platform has no content without events, and search, feed, and ticketing all depend on listings existing.

Expected Deliverable: An event creation form that saves and publishes events.

Acceptance Criteria:

- Given an organizer is logged in, when they open "Create Event", then the system displays a form for title, description, date, time, location, image, and ticket options.
- Given the organizer submits a form with all required fields completed, when the event is saved, then the system publishes the event and shows it on the organizer's page.
- Given required fields are missing, when the organizer submits, then the system blocks submission and highlights the missing fields.

US-03 – Edit or Cancel Event

User Story: As an event organizer, I want to edit or cancel my event, so that I can keep attendees updated when plans change.

Priority: Medium

Priority Justification: Important for real-world use and trust, but the platform can launch and function with event creation alone.

Expected Deliverable: Edit and cancel controls on the organizer's event management page.

Acceptance Criteria:

- Given an organizer views their own event, when they select "Edit", then the system lets them update event details and save changes.
- Given an organizer selects "Cancel Event", when they confirm, then the system marks the event as cancelled.
- Given an event is cancelled, when registered attendees are affected, then the system notifies them and initiates refunds for paid tickets.

US-04 – Personalized Event Feed

User Story: As an attendee, I want to see a feed of events recommended to me, so that I can discover events I'm likely to enjoy.

Priority: Medium

Priority Justification: The recommendation algorithm is a key differentiator for Who's In, but users can still find events through search without it.

Expected Deliverable: A home feed ranked by a recommendation algorithm using interests, location, past activity, and engagement.

Acceptance Criteria:

- Given a logged-in user opens the home page, when the feed loads, then the system displays events ranked by relevance to that user.
- Given a user likes, saves, or registers for events, when they next refresh the feed, then the system reflects those interactions in the ranking.
- Given a new user with no activity, when the feed loads, then the system shows popular and nearby events as a default.

US-05 – Search Events by Location and Keyword

User Story: As an attendee, I want to search for events happening in my area, so that I can find something to attend nearby.

Priority: High

Priority Justification: Search is the main way users find events and is a core planned API integration.

Expected Deliverable: A search feature integrated with a search/location API, with filters for

date, category, distance, and price.

Acceptance Criteria:

- Given a user enters a keyword or location, when they submit the search, then the system displays matching events.
- Given search results are displayed, when the user applies filters (date, category, distance, price), then the system updates the results accordingly.
- Given no events match the search, when results load, then the system shows a "no results" message with suggested alternatives.

US-06 – Register for Free Event

User Story: As an attendee, I want to register for a free event, so that I can reserve my spot.

Priority: High

Priority Justification: Registration is a core purpose of the app and the simplest path from discovering an event to attending it.

Expected Deliverable: A registration function that confirms attendance and generates a ticket.

Acceptance Criteria:

- Given a logged-in user views a free event with availability, when they select "Register", then the system confirms their registration.
- Given the registration succeeds, when it is processed, then the system issues a digital ticket and sends a confirmation.
- Given the user is already registered, when they view the event, then the system shows their registered status instead of the register button.

US-07 – Purchase Tickets with Stripe

User Story: As an attendee, I want to buy tickets securely online, so that I can guarantee entry to a paid event.

Priority: High

Priority Justification: Ticket sales are the core transaction and revenue driver, and payments carry security and financial risk, so they must work correctly from release.

Expected Deliverable: A checkout flow integrated with the Stripe API supporting ticket selection, payment, and confirmation.

Acceptance Criteria:

- Given a user selects a ticket type and quantity, when they proceed to checkout, then the system displays an order summary with the total price.
- Given the user enters valid payment details, when Stripe confirms payment, then the system issues tickets and sends a receipt.

- Given payment fails or is declined, when the result is returned, then the system shows an error and does not issue tickets.

US-08 – View and Present My Tickets

User Story: As an attendee, I want to access my tickets in the app, so that I can show them at the event entrance.

Priority: Medium

Priority Justification: Users need access to their tickets, but a confirmation email can serve as a temporary substitute early on.

Expected Deliverable: A "My Tickets" page with a scannable QR code for each ticket.

Acceptance Criteria:

- Given a user has purchased or registered for events, when they open "My Tickets", then the system lists all their upcoming tickets.
- Given a ticket is selected, when the detail view opens, then the system displays a unique QR code and event information.
- Given an event has passed, when the user views tickets, then the system moves it to a "Past" section.

US-09 – Cancel Registration / Request Refund

User Story: As an attendee, I want to cancel my registration, so that I can change my plans when necessary.

Priority: Medium

Priority Justification: It supports user trust and fair refund handling, but it depends on registration and payment working first.

Expected Deliverable: A cancellation function accessible from the My Tickets page, with refunds processed through Stripe where eligible.

Acceptance Criteria:

- Given a user has an eligible upcoming ticket, when they view it, then the system displays a cancel option.
- Given the user selects "Cancel," when prompted, then the system asks for confirmation.
- Given the user confirms, when processing succeeds, then the system marks the ticket as cancelled, releases the spot, and issues a refund if the refund policy applies.

US-10 – Share Events

User Story: As a user, I want to share events with friends and on social platforms, so that more people can discover and attend them.

Priority: Medium

Priority Justification: Sharing drives growth and the social side of the app, but the core booking flow works without it.

Expected Deliverable: A share function generating a link and in-app share options.

Acceptance Criteria:

- Given a user views an event, when they select "Share," then the system displays sharing options (copy link, in-app post, external platforms).
- Given a user shares an event in-app, when the post is published, then the system displays it on their profile and their followers' feeds.
- Given someone opens a shared link, when the page loads, then the system displays the correct event.

US-11 – Add Event to Calendar

User Story: As an attendee, I want to add events to my calendar, so that I don't forget when and where they happen.

Priority: Low

Priority Justification: It is a convenience feature that improves the experience but isn't needed for the platform to function, so it can come after core features.

Expected Deliverable: A calendar integration (e.g., Google Calendar API or .ics export) available on event and ticket pages.

Acceptance Criteria:

- Given a user has a ticket or saved event, when they select "Add to Calendar," then the system creates a calendar entry with the correct title, date, time, and location.
- Given the user has connected their calendar account, when the event is added, then the system confirms success.
- Given the event time or location changes, when the organizer updates it, then the system notifies the user and updates the calendar entry (or prompts them to).

US-12 – Notifications and Reminders

User Story: As an attendee, I want to receive reminders and updates, so that I stay informed about my events.

Priority: Medium

Priority Justification: It keeps attendees informed about confirmations and changes, but it supports the core flow rather than being part of it.

Expected Deliverable: Email and in-app notifications for confirmations, reminders, and changes.

Acceptance Criteria:

- Given a user registers or buys a ticket, when the transaction completes, then the system sends a confirmation notification.
- Given an event is within 24 hours, when the reminder time is reached, then the system sends a reminder to registered attendees.
- Given an organizer changes or cancels an event, when the update is saved, then the system notifies all registered attendees.

5. Estimation and Prioritization

Each user story was estimated using the required scale of **1, 2, 3, 5, and 8 story points**. The points represent relative effort rather than the exact number of hours required. When estimating each story, the team considered the amount of development work, technical difficulty, testing requirements, external integrations, and uncertainty involved.

The stories were prioritized using the required levels of **Blocker, Critical, Major, Minor, and Trivial**. The priority of a story was based on its value to users, its dependencies, and whether other features could be developed without it.

ID	User Story	Story Points	Priority
US-01	Create Account and Log In	5	Blocker
US-02	Create Event	5	Blocker
US-03	Edit or Cancel Event	5	Major
US-04	Personalized Event Feed	8	Major
US-05	Search Events by Location and Keyword	5	Blocker
US-06	Register for Free Event	5	Critical
US-07	Purchase Tickets with Stripe	8	Critical
US-08	View and Present My Tickets	3	Major
US-09	Cancel Registration or Request Refund	5	Major
US-10	Share Events	5	Major
US-11	Add Event to Calendar	3	Minor

US-12	Notifications and Reminders	5	Major
-------	-----------------------------	---	-------

Estimation

The story-point estimates are based on relative effort, technical complexity, testing requirements, dependencies, and uncertainty. They are not intended to represent hours or exact completion times.

US-04 and **US-07** received the highest estimate of **eight** story points. The personalized event feed requires ranking logic that considers interests, location, past activity, and engagement. It must also provide reasonable recommendations for new users who have little or no activity history. **US-07** involves Stripe integration, payment verification, security considerations, failed transactions, and the requirement that tickets are issued only after a successful payment. These factors make both stories more complex and uncertain than the others.

US-08 and **US-11** were estimated at **three** points because their initial implementations are more contained. For **US-08**, the registration and payment features already create the ticket records. This story mainly retrieves those records and displays the relevant event information and QR code. **US-11** can initially be implemented using an .ics file calendar file rather than requiring a complete integration with an external calendar service. This still allows users to add events to their calendars while keeping the first version manageable.

The remaining stories **US-01**, **US-02**, **US-03**, **US-05**, **US-06**, **US-09**, **US-10** and **US-12** received **five points** because each requires a moderate amount of work across the interface, backend, database, and testing. For example, account access requires secure password handling and session management, while event creation requires input validation, database storage, and publishing. Search must support keywords, locations, filters, and empty-result handling. Registration, refunds, sharing, and notifications also require several connected operations, but their expected workflows are better understood than the recommendation and payment features.

We did not give any stories **one** or **two** points because each story represents a complete feature that requires several development tasks. Smaller point values may be used later when these features are broken down into individual tasks during sprint planning.

Prioritization

US-01, **US-02**, and **US-05** are classified as **Blockers** because they establish the basic working structure of Whozin. Users need to create accounts and log in, organizers need to publish events,

and attendees need a way to search for those events. Most of the remaining stories depend on these capabilities, so they should be addressed first.

US-06 and **US-07** are **Critical** because registration and ticket purchasing connect event discovery to actual attendance. Free-event registration provides the simpler registration path, while Stripe purchasing supports paid events. Both are important early features, but they depend on accounts and event listings already being available.

US-03, **US-04**, **US-08**, **US-09**, **US-10**, and **US-12** are **Major** priorities. Editing or cancelling events, accessing tickets, requesting refunds, sharing events, and receiving notifications are all important to the overall user experience. However, the initial account, event, search, and registration flows can still be developed before these features are completed. The personalized feed is also major because it is an important part of the Whozin concept, but users can temporarily discover events through search and filters.

US-11 is given a **Minor** priority because adding events to a calendar is helpful, but the main features of Whozin can still work without it.

Users can still find events, register, buy tickets, and view their tickets. None of the current stories are considered trivial because each one adds a useful function to the platform. The team may adjust these estimates and priorities later as we gain more experience with the system, complete sprints, and receive feedback.

6. Initial Product Backlog

The product backlog organizes all twelve user stories from Section 4 in the order the team intends to develop them. Story points and priority levels match the estimates in Section 5, and each story is linked to the functional requirement in Section 2 that it implements, so every requirement can be traced to a backlog item. Stories are ordered by priority first. When stories share the same priority, they are ordered by dependencies, then by user value, technical risk, and development needs.

Order	ID	User Story / Short Title	Priority	Story Points	Status	Related FR	Depends On
-------	----	--------------------------	----------	--------------	--------	------------	------------

1	US-01	Create Account and Log In	Blocker	5	To Do	FR-01	None
2	US-02	Create Event	Blocker	5	To Do	FR-02	US-01
3	US-05	Search Events by Location and Keyword	Blocker	5	To Do	FR-05	US-02
4	US-06	Register for Free Event	Critical	5	To Do	FR-06	US-01, US-02
5	US-07	Purchase Tickets with Stripe	Critical	8	To Do	FR-07	US-06
6	US-08	View and Present My Tickets	Major	3	To Do	FR-08	US-06
7	US-12	Notifications and Reminders	Major	5	To Do	FR-12	US-06
8	US-09	Cancel Registration or Request Refund	Major	5	To Do	FR-09	US-07, US-08
9	US-03	Edit or Cancel Event	Major	5	To Do	FR-03	US-02, US-09, US-12
10	US-10	Share Events	Major	5	To Do	FR-10	US-02
11	US-04	Personalized Event Feed	Major	8	To Do	FR-04	US-05, US-06
12	US-11	Add Event to Calendar	Minor	3	To Do	FR-11	US-08, US-12

Backlog Summary

Priority	Stories	Story Points
Blocker	US-01, US-02, US-05	15
Critical	US-06, US-07	13
Major	US-08, US-12, US-09, US-03, US-10, US-04	31
Minor	US-11	3
Total	12 stories	62

Ordering Within Each Priority Level

Blocker. US-01 is first because every protected feature, including ticketing, the feed, and organizer tools, requires an authenticated user. US-02 follows because search, registration, and ticketing cannot work until events exist. US-05 completes the Blocker group by giving attendees a way to find published events.

Critical. Free-event registration (US-06) comes before Stripe purchasing (US-07). US-06 builds the ticket-issuing and capacity logic in a simpler, lower-risk flow, and US-07 then reuses that logic while adding payment verification. This also lets the team meet NFR-05 (no duplicate tickets or overselling) before external payments are introduced.

Major. These six stories are ordered by their dependencies:

- US-08 is first because users need to see tickets as soon as they are issued, and refunds are started from the My Tickets page.
- US-12 follows because both cancellation stories must notify attendees.
- US-09 builds the Stripe refund process.
- US-03 comes after US-09 and US-12 because cancelling an event must notify registered attendees and refund paid tickets.
- US-10 has few dependencies, so it can fill spare capacity in any sprint.

- US-04 is last in this group. It carries the most uncertainty, and its ranking can only be tested properly once registration and activity data exist.

Minor. US-11 builds on ticket pages and change notifications. It is the first story to defer if the team's velocity is lower than planned.

Items Pending Refinement

The following items come from the project proposal or are referenced by current stories but do not yet have their own user story. They are recorded here so they remain traceable. Each will be written as a user story with acceptance criteria, estimated, and prioritized during sprint planning before any development begins.

ID	Backlog Item	Why It Is Needed	Status
BI-01	Save or bookmark events	Proposal core feature; referenced in US-01, US-04, US-11, and FR-11	Needs refinement
BI-02	Follow organizers and content creators	Proposal core feature; Section 1 content creators; US-10 shares posts to followers' feeds	Needs refinement
BI-03	Like and comment on event posts	Proposal core feature; US-04 ranks events using likes	Needs refinement
BI-04	Publish promotional posts and short videos	Proposal core feature; Section 1 content creators share event content and collaborate with organizers	Needs refinement
BI-05	Organizer attendee and engagement view	Proposal organizer role: view registered attendees, ticket information, and engagement	Needs refinement
BI-06	Administrator moderation and user management	Section 1 administrator role (accounts, permissions, reports, content); NFR-03 requires protected administrator functions	Needs refinement

Backlog Maintenance

The backlog is a living engineering artifact. It is maintained as GitHub Issues in the project repository (<https://github.com/JINAL-1/Whozin>), with labels for priority and story points. The team will review it at each sprint planning meeting and sprint review, where it will:

- update each story's status (To Do, In Progress, Done)
- re-estimate stories through planning poker as details become clearer
- refine and add the pending items above
- reorder items when requirements, risks, or feedback change

Unfinished sprint work returns to the backlog and is re-prioritized rather than extending the sprint. The backlog will be updated at every later project milestone.

7. Initial Iteration Plan

Time : 2 weeks

Sprint Goal : Establish basic functionality

User Stories covered : US-1, US-2, US-5

Overall Priority type : Blocker

Story points covered : 15

Expectation : Two Functionality flows : User and Organizer and establish US-1,2 and 5. users will be able to create an account, reset their password, and search for events, while organizers can create an account and post events.

USER ITERATION PLAN

US-01 CREATE ACCOUNT AND LOGIN

US-05 SEARCH EVENTS BY LOCATION AND KEYWORDS

Task no.	User Story Id	Task	Team member assigned
1	US-01	Create sign up page	Jinal
2	US-01	Create user database	Jinal
3	US-01	Connect sign up credentials to database	Jinal
4	US-01	Create forgot password page	Pranav
5	US-01	Create password reset function	Pranav
6	US-01	Update password on database	Pranav
7	US-05	Create event search function	Dean
8	US-05	Connect event search to database	Dean
9	US-05	Display search result	Dean
10	US-01/05	Test user account and event search functionality	Jinal and Pranav

ORGANISER ITERATION PLAN

US-02 CREATE EVENT

Task no.	User Story ID	Task	Team member assigned
1	US-02	Create event page	Julia

2	US-02	Connect event details to database	Julia
3	US-01	Create organizer sign up	Krishi
4	US-01	Connect organizer credentials and details to database	Krishi
5	US-01	Create Forgot password page for organiser	Krishi
6	US-01	Update organizer password on database	Krishi
7	US-02	Check connection between Organiser account and created event	Julia
8	US-01	Test organiser and event creation functionality	Julia and Dean

8. Project Risks and Important Assumptions

Project Risks

Risk no.	Risk	Likelihood	Impact	How it could affect	Mitigation Strategy
1	Changing deadlines or priorities	High	High	Requirements, deadlines, or priorities may change during development, causing planned work to be delayed or replaced.	Reviewing priorities regularly, complete important tasks first, and move lower-priority work to later sprints.
2	Time management	High	High	Coding, debugging,	Breaking work into

	and underestimated tasks			database integration, testing, or learning new techniques may take longer than expected.	smaller tasks, estimate realistically, track progress, and start difficult tasks early.
3	Testing delay	High	High	If testing is delayed until late in the sprint or just before submission, major bugs tend to show up all at once. That can cause last-minute rework, stress, broken features, or even incomplete deliverables. It also becomes harder to figure out what caused the bugs, because everything's being merged and changed at once.	Testing each feature as it is developed, run automated tests before merging code, and perform integration testing regularly so major bugs are found early.
4	Learning new languages, tools, or techniques	Medium	High	Team members may need to learn unfamiliar languages, frameworks, APIs, database techniques, or development methods. This could reduce performance and take time away from implementation.	Identifying learning needs early, using small prototypes, sharing knowledge between members, and allowing learning time when estimating tasks.
5	Unexpected costs or resource limitations	Medium	Medium	Hosting, databases, maps, notifications, payment services, or other tools may have costs or usage limits that affect development.	Use free or student resources where possible, monitor usage, and identify alternative tools before relying on paid services.

Assumptions

- Whozin will initially focus on events in Thunder Bay, with enough event information available for users to search and explore.
- External services such as payments, maps, and notifications will be available during development and testing.
- Payments will be handled through a third-party payment service, so Whozin will not store users' complete card information.
- The first version will support basic ticket purchasing and digital tickets without trying to provide every feature of a commercial ticketing platform.
- Event recommendations will initially be based on user interests and activity, without requiring advanced machine learning.
- Users will have internet access and a supported mobile device while using Whozin.